

Week 5 Quiz Review

Python data exploration and wrangling with pandas

Every output below was produced by actually running the code. Two datasets appear: the lecture's **dodgers** and **usedcars** files, and a tiny made-up **cafe** table (defined in section 1) so you can see every result at a glance. The "From R" notes mark where pandas differs from the tidyverse.

0. The big picture

The week 5 notebook follows one workflow on a DataFrame:

Step	Question	pandas tools
Load	Get the data in	<code>pd.read_csv(...)</code>
Inspect	What do I have?	<code>.head(), .info(), .dtypes,</code> <code>.columns, .shape</code>
Summarize	What do the values look like?	<code>.describe(), .min(),</code> <code>.median(), .value_counts(),</code> <code>.nunique()</code>
Plot	What is the shape?	<code>sns.histplot, sns.boxplot,</code> <code>sns.barplot</code>
Sort / filter / select	Which rows and columns?	<code>.sort_values(), .isin(),</code> <code>df[cond], df[[cols]], .iloc</code>
Group	Summaries per category	<code>.groupby().agg(),</code> <code>.transform()</code>
Missing data	What is absent?	<code>.isna().sum(), .fillna()</code>
New columns	Derive something	<code>df["new"] = ...,</code> <code>.str.contains(), .astype()</code>

The quiz tests code and output together, so the most valuable habit is: for every line, say out loud *what type of thing it returns* (a Series, a DataFrame, a number, or None) and *what its index looks like*.

1. Setup and the cafe table

```
import pandas as pd
import numpy as np
```

Loading (the notebook uses a GitHub raw URL; a local file works the same way):

```
dodgers = pd.read_csv(url + "dodgers.csv")      # url is a string ending in "/"
usedcars = pd.read_csv(url + "usedcars.csv")
```

%whos is a Jupyter/Colab **magic command**, not Python. It lists your variables with type and size. It only works inside a notebook.

The cafe table used for all small examples (None becomes NaN, which forces those columns to float64):

```
cafe = pd.DataFrame({
    "store": ["North", "North", "South", "South", "South", "East", "East"],
    "drink": ["Latte", "Mocha", "Latte", "Tea", "Mocha", "Tea", "Latte"],
    "price": [5, 6, 5, 3, 6, 3, None],
    "cups": [10, 4, 8, 12, None, 7, 9]
})
```

cafe

	store	drink	price	cups
0	North	Latte	5.0	10.0
1	North	Mocha	6.0	4.0
2	South	Latte	5.0	8.0
3	South	Tea	3.0	12.0
4	South	Mocha	6.0	NaN
5	East	Tea	3.0	7.0
6	East	Latte	NaN	9.0

2. Inspecting a DataFrame

cafe.shape

(7, 4)

cafe.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7 entries, 0 to 6
Data columns (total 4 columns):
#   Column  Non-Null Count  Dtype
---  -
0   store   7 non-null        object
1   drink   7 non-null        object
2   price   6 non-null        float64
3   cups    6 non-null        float64
dtypes: float64(2), object(2)
memory usage: 356.0+ bytes
```

cafe.dtypes

```
store    object
drink     object
price    float64
cups     float64
dtype: object
```

- `.info()` prints (it returns None): index range, each column's **Non-Null Count** (that is how you spot missing data) and Dtype.
- `.dtypes` is an **attribute** (no parentheses) and returns a Series.
- `.shape` is also an attribute: (rows, columns).
- Text columns show as object. A column with missing values in a number column becomes float64, because NaN is a float.
- Change a type with `df["col"] = df["col"].astype(int)`. You must assign it back. It **fails on NaN** (`IntCastingNaNError`), so fill or drop first.

Looking at columns

```
cafe["price"]           # ONE name -> a Series
```

```
0    5.0
1    6.0
2    5.0
3    3.0
4    6.0
5    3.0
6    NaN
Name: price, dtype: float64
```

```
cafe[["store", "price"]] # a LIST of names -> a DataFrame
```

```
   store  price
0  North   5.0
1  North   6.0
2  South   5.0
3  South   3.0
4  South   6.0
5   East   3.0
6   East   NaN
```

The double brackets are a list inside the selector brackets. `cafe["store", "price"]` (a comma, no list) is a **KeyError**.

From R: every column name in pandas is a **quoted string** ("attend"), unlike tidyverse verbs where you write `select(attend)` bare.

3. Summary statistics

```
cafe.describe()
```

```
count    price    cups
count  6.000000  6.000000
mean    4.666667  8.333333
std      1.366260  2.732520
min      3.000000  4.000000
25%      3.500000  7.250000
50%      5.000000  8.500000
75%      5.750000  9.750000
max      6.000000 12.000000
```

- `describe()` only summarizes **numeric** columns by default. `count` is non-missing values per column (6, not 7: the NaN is skipped).
- If a number column were stored as text (object), `describe()` would show `count / unique / top / freq` instead. That is the signal to check dtypes.

Single statistics (all skip NaN by default):

```
cafe["price"].min(), cafe["price"].max(), cafe["price"].median()
```

```
(3.0, 6.0, 5.0)
```

```
cafe["price"].agg(["mean", "median"])
```

```
mean      4.666667
median    5.000000
Name: price, dtype: float64
```

Counting categories

```
cafe["drink"].value_counts()
```

```
drink
Latte    3
Mocha    2
Tea       2
Name: count, dtype: int64
```

```
cafe.nunique()
```

```
store      3
drink       3
price       3
cups        6
dtype: int64
```

- `value_counts("col")` and `df["col"].value_counts()` give the same counts, sorted largest first.
- `nunique()` = number of **distinct non-missing** values per column (the notebook's "unique job titles" example).
- `df.value_counts()` with **no argument** counts duplicate *rows* ("good to find duplicates"). It also **drops rows containing NaN** by default:

```
cafe.value_counts()
```

```
store  drink  price  cups
East   Tea    3.0    7.0    1
North  Latte   5.0   10.0    1
        Mocha   6.0    4.0    1
South  Latte   5.0    8.0    1
        Tea    3.0   12.0    1
Name: count, dtype: int64
```

Seven rows went in and only five come out, each appearing once.

4. Quick plots

```
import seaborn as sns
import matplotlib.pyplot as plt
```

Code	Draws
<code>sns.histplot(data=dodgers, x="day_of_week")</code>	counts of a categorical variable (bars per category)
<code>sns.histplot(data=dodgers, x="attend", binwidth=5000)</code>	distribution of a number; each bar 5,000 wide
<code>sns.boxplot(data=dodgers, x="attend")</code>	one horizontal box: median, quartiles, outliers
<code>sns.boxplot(data=dodgers, x="day_of_week", y="attend")</code>	one box per day
<code>sns.boxplot(data=dodgers, y="day_of_week", x="attend")</code>	same but horizontal: swap x and y
<code>sns.barplot(data=dodgers, x="day_of_week", y="attend")</code>	bar = mean of y per x (with an error bar)

- Seaborn takes the DataFrame as `data=` and the **column names as strings** for `x= / y=`.
- Matplotlib adds labels afterwards: `plt.xlabel("Attendance"), plt.ylabel(...), plt.title(...), plt.show()`.
- `binwidth=` sets how wide each bar is; `bins=` would set how **many** bars.
- `plt.savefig('fig1.png')` **silently overwrites** an existing file. The lecture flags this with “BE VERY careful”.
- Box plot reading: the line in the box is the median; the box is the middle 50% (25th to 75th percentile); dots beyond the whiskers are outliers. The notebook prints min, max and median right under one to compare.

5. Sorting

```
cafe.sort_values("price")
```

```
   store  drink  price  cups
3  South   Tea    3.0   12.0
5   East   Tea    3.0    7.0
0 North  Latte    5.0   10.0
2  South  Latte    5.0    8.0
1 North  Mocha    6.0    4.0
4  South  Mocha    6.0   NaN
6   East  Latte   NaN    9.0
```

```
cafe.sort_values("price", ascending=False)
```

```
   store  drink  price  cups
1 North  Mocha    6.0    4.0
4  South  Mocha    6.0   NaN
0 North  Latte    5.0   10.0
2  South  Latte    5.0    8.0
3  South   Tea    3.0   12.0
5   East   Tea    3.0    7.0
6   East  Latte   NaN    9.0
```

- Default is **smallest to largest**. `ascending=False` reverses it.
- **NaN is always placed last**, in either direction.
- The index labels travel with their rows (note 0..6 are shuffled).
- The original is **not** changed. To change it: assign the result, or `inplace=True`, which **returns None**:

```
r = cafe.sort_values("cups", inplace=True)
print(r)
cafe.head(3)
```

None

	store	drink	price	cups
1	North	Mocha	6.0	4.0
5	East	Tea	3.0	7.0
2	South	Latte	5.0	8.0

Trap: after `inplace=True` the whole DataFrame is reordered, so *later* cells see a different row order (and different index labels) than earlier ones. The notebook sorts dodgers in place in the middle of the lecture.

6. Filtering rows

Step 1: make a True/False Series.

```
cafe["drink"].isin(["Tea", "Mocha"])
```

```
0    False
1     True
2    False
3     True
4     True
5     True
6    False
Name: drink, dtype: bool
```

```
~cafe["drink"].isin(["Tea", "Mocha"])
```

```
0     True
1    False
2     True
3    False
4    False
5    False
6     True
Name: drink, dtype: bool
```

`.isin([...])` is R's `%in%`. Tilde `~` flips True/False ("NOT in"). Python's `not` does **not** work on a Series.

Step 2: put it inside `df[ ... ]`.

```
cafe[cafe["drink"].isin(["Latte"])]
```

	store	drink	price	cups
0	North	Latte	5.0	10.0
2	South	Latte	5.0	8.0
6	East	Latte	NaN	9.0

```
cafe[cafe["price"] >= 5]
```

	store	drink	price	cups
0	North	Latte	5.0	10.0
1	North	Mocha	6.0	4.0
2	South	Latte	5.0	8.0
4	South	Mocha	6.0	NaN

- **The index keeps the original labels** (0, 1, 2, 4 - label 3 is gone). This is the lecture's "Index: 43 entries, 0 to 79 ... why?" point: 43 rows, but the labels span the original range. The NaN price fails ≥ 5 and is dropped.
- To renumber: `.reset_index(drop=True)`. Without `drop=True` the old index becomes a new column named `index`.

```
cafe[cafe["price"] >= 5].reset_index(drop=True)
```

	store	drink	price	cups
0	North	Latte	5.0	10.0
1	North	Mocha	6.0	4.0
2	South	Latte	5.0	8.0
3	South	Mocha	6.0	NaN

Combining conditions: `&` (and), `|` (or), `~` (not), **each condition in parentheses**. The words and / or fail with "truth value of a Series is ambiguous".

```
cafe[(cafe["store"] == "South") & (cafe["price"] > 3)]
```

	store	drink	price	cups
2	South	Latte	5.0	8.0
4	South	Mocha	6.0	NaN

```
cafe[(cafe["store"] == "East") | (cafe["price"] > 5)]
```

	store	drink	price	cups
1	North	Mocha	6.0	4.0
4	South	Mocha	6.0	NaN
5	East	Tea	3.0	7.0
6	East	Latte	NaN	9.0

Note row 6 (East, price NaN) is kept by the `|` because `store == "East"` is True; `NaN > 5` is False.

The notebook's examples, with the real data:

```
dodgersSS = dodgers[dodgers["day_of_week"].isin(["Saturday", "Sunday"])]  
dodgersSS.shape
```

```
(26, 12)
```

```
dodgers40K = dodgers[dodgers["attend"] >= 40000]  
dodgers40K.shape
```

```
(43, 12)
```

7. Selecting columns and positions

Three ways to keep columns: a list of names, a stored list, or positions.

```
cols_keep = ["store", "cups"]
cafe[cols_keep]
```

	store	cups
0	North	10.0
1	North	4.0
2	South	8.0
3	South	12.0
4	South	NaN
5	East	7.0
6	East	9.0

`.iloc[rows, cols]` selects by **position** (0-based), and a slice's stop is **excluded**. Think “matrix notation”.

```
cafe.iloc[1:4, [0, 2]]
```

	store	price
1	North	6.0
2	South	5.0
3	South	3.0

```
cafe.iloc[:, [0, 2]]
```

	store	price
0	North	5.0
1	North	6.0
2	South	5.0
3	South	3.0
4	South	6.0
5	East	3.0
6	East	NaN

```
cafe.sort_values("cups", ascending=False).iloc[0:2, [0, 1]]
```

	store	drink
3	South	Tea
0	North	Latte

- `iloc[1:4, [0, 2]]` = rows at positions 1, 2, 3 and the columns at positions 0 and 2 (store, price).
- `:` alone means “all”.
- The last example is the notebook’s pattern “top 3 attended games”: **sort first, then slice**. `.iloc` counts positions *in the current order*, but the index labels (3, 0 here) show where the rows came from.
- Use `dodgers.columns` to see which position is which column.

Mini-check: in `dodgers`, `iloc[0:3, [0, 1, 2, 4]]` after sorting by `attend` descending gives month, day, attend and **opponent** (position 4), because `day_of_week` is position 3.

8. Group summaries

`groupby("col")` splits the data; an aggregating function summarizes each piece. Available: `sum`, `min`, `max`, `count`, `var`, `std`, `median`, `mean`.

One column:

```
cafe.groupby("store")["cups"].mean()
```

```
store
East      8.0
North     7.0
South    10.0
Name: cups, dtype: float64
```

Order of the pieces: `df.groupby("group_col")["value_col"].function()`. Result: a Series indexed by the group values (sorted alphabetically). **Skips NaN.**

Why dodgers.groupby("day_of_week").mean() errors: it tries to average *every* column including text columns (object), which fails:

```
cafe.groupby("store").mean()
```

```
TypeError: agg function failed [how->mean,dtype->object]
```

The catch in the notebook: `.mean("attend")` does **not** select a column. The first positional argument of `mean` is `numeric_only`, and a non-empty string is truthy, so it means “only numeric columns”. It returns the mean of *all* numeric columns:

```
dodgers.groupby("day_of_week").mean("attend").head(3)
```

	day	attend	temp
day_of_week			
Friday	19.307692	40116.923077	69.692308
Monday	13.416667	34965.666667	72.833333
Saturday	17.923077	43072.923077	72.692308

Counting: `count()` counts **non-missing** values; `size()` counts rows.

```
cafe.groupby("store")["cups"].count()
```

```
store
East      2
North     2
South     2
Name: cups, dtype: int64
```

```
cafe.groupby("store")["cups"].size()
```

```
store
East      2
North     2
South     3
Name: cups, dtype: int64
```

.agg() - several statistics at once

A list of function names (as strings):

```
cafe["price"].agg(["mean", "median"])
```

```
mean      4.666667
median    5.000000
Name: price, dtype: float64
```

```
cafe.groupby("store")["cups"].agg(["sum", "mean"])
```

```
      sum  mean
store
East   16.0   8.0
North  14.0   7.0
South  20.0  10.0
```

Two columns + a list of functions gives **two-level column headers**:

```
cafe.groupby("store")[["price", "cups"]].agg(["mean", "max"])
```

```
      price      cups
      mean  max  mean  max
store
East  3.000000  3.0   8.0   9.0
North  5.500000  6.0   7.0  10.0
South  4.666667  6.0  10.0  12.0
```

Single brackets ["cups"] = Series in, flat headers out. Double brackets [["cups"]] = DataFrame in, two-level headers out.

Named aggregation gives your own column names: new_name=("column", "function"):

```
cafe.groupby("drink").agg(total_cups=("cups", "sum"), avg_price=("price", "mean"))
```

```
      total_cups  avg_price
drink
Latte          27.0         5.0
Mocha           4.0         6.0
Tea            19.0         3.0
```

On a single selected column you can drop the tuple: .agg(mean_attend="mean", median_attend="median").

.agg() vs .transform()

	agg	transform
Rows returned	one per group	same as the original
Index	the group labels	the original index

	agg	transform
Use for	summary tables	adding a group statistic as a new column

```
cafe["store_avg"] = cafe.groupby("store")["price"].transform("mean")
cafe[["store", "price", "store_avg"]]
```

```
   store  price  store_avg
0  North    5.0    5.500000
1  North    6.0    5.500000
2  South    5.0    4.666667
3  South    3.0    4.666667
4  South    6.0    4.666667
5   East    3.0    3.000000
6   East   NaN    3.000000
```

The group mean is repeated on every row of that group (South 4.666667 three times). The notebook's practice: add the mean price and the count of cars by color to usedcars:

```
usedcars['mean_price_by_color'] = usedcars.groupby('color')['price'].transform('mean')
usedcars['count_by_color']      = usedcars.groupby('color')['price'].transform('count')
```

Trap: assigning an agg result as a column aligns on the index and produces all NaN, because the group labels do not match 0, 1, 2, ...

9. Missing data

```
cafe.isna()
```

```
   store  drink  price  cups
0  False  False  False  False
1  False  False  False  False
2  False  False  False  False
3  False  False  False  False
4  False  False  False   True
5  False  False  False  False
6  False  False   True  False
```

```
cafe.isna().sum()
```

```
store    0
drink    0
price    1
cups     1
dtype: int64
```

- `.info()` (Non-Null Count lower than the row count) and `.isna()` both detect missing values. `~df.isna()` is the reverse (True where present).
- **Chaining** `.isna().sum()` counts missing values **per column**. `.isna().count()` would just count rows (7 everywhere): a common wrong answer.

- `isna().mean()` gives the **fraction** missing.

```
cafe["price"].isna().mean()
```

```
0.14285714285714285
```

Filling with a statistic:

```
cafe["price"].fillna(cafe["price"].mean())
```

```
0    5.000000
1    6.000000
2    5.000000
3    3.000000
4    6.000000
5    3.000000
6    4.666667
Name: price, dtype: float64
```

```
cafe["price"].fillna(cafe["price"].mean())
cafe["price"].isna().sum()
```

```
1
```

`fillna` returns a **new** Series; the original still has its NaN (the second example prints 1). To keep it, assign: `cafe["price"] = cafe["price"].fillna(...)`.

Dropping is the other option the lecture mentions ("just drop the observations with missing data"): `.dropna()` removes every row with any NaN.

```
cafe.dropna().shape
```

```
(5, 4)
```

10. Strings and new columns

`.str.contains("pattern")` tests each string and returns True/False. The pattern is a **regular expression**: `|` means OR. It is **case-sensitive**.

```
cafe["drink"].str.contains("t")
```

```
0    True
1   False
2    True
3   False
4   False
5   False
6    True
Name: drink, dtype: bool
```

```
cafe["drink"].str.contains("Latte|Tea")
```

```
0    True
1   False
2    True
3    True
4   False
5    True
6    True
Name: drink, dtype: bool
```

- “Tea” has a capital T, so contains("t") is False for it.
- .str is required: cafe["drink"].contains(...) is an AttributeError.
- Lecture example: str.contains("S") on day_of_week matches Saturday and Sunday (capital S only), exactly like the isin(["Saturday", "Sunday"]) filter (26 games either way). "Saturday|Sunday" is the explicit version.
- .astype(int) turns True/False into 1/0, and .sum() counts the matches:

```
cafe["drink"].str.contains("Latte|Tea").astype(int).sum()
```

5

Adding a column is plain assignment with the new name in quotes: df["new_col"] = <Series of the same length>. The lecture builds weekend_game (True/False) and then is_weekend (1/0 with .astype(int)).

```
dodgers['is_weekend'] = (
    dodgers["day_of_week"]
    .str.contains("Saturday|Sunday")
    .astype(int)
)
```

The parentheses around a chain let you break it over several lines. Data cleaning aside (not in the notebook): some dodgers values have a trailing space ("Clear "), which is why .str.strip() exists.

11. Lecture practice problems, solved (real data)

The notebook leaves these blank. Here are working answers, run on usedcars.csv (150 rows).

a) Missing data? Types? Categorical variables? usedcars.info() shows 150 non-null in every column; year, price, mileage are int64; model, color, transmission are object (categorical). usedcars.isna().sum() is all zeros.

b) Histogram of price: sns.histplot(data=usedcars, x="price").

c) No Black cars, sorted by price (low to high):

```
noblack = usedcars[usedcars["color"] != "Black"].sort_values("price")
print(noblack.shape)
noblack.head(3)
```

```
(115, 6)
   year  model  price  mileage  color transmission
149  2000    SE   3800   109259    Red          AUTO
148  2001    SE   4899   151479  Yellow          AUTO
147  2000    SE   5980    96841    Red          AUTO
```

d) 3 most expensive white + 3 least expensive blue: two filtered/sorted pieces, then stack them (`pd.concat` is not in the notebook but is the natural tool):

```
white3 = usedcars[usedcars["color"] == "White"].sort_values("price", ascending=False).head(3)
blue3 = usedcars[usedcars["color"] == "Blue"].sort_values("price").head(3)
pd.concat([white3, blue3])
```

	year	model	price	mileage	color	transmission
4	2012	SE	17500	8367	White	AUTO
14	2011	SE	16000	15367	White	AUTO
21	2010	SEL	15980	39943	White	AUTO
127	2007	SE	9651	63296	Blue	AUTO
112	2005	SES	10995	69415	Blue	AUTO
102	2009	SE	11999	36124	Blue	AUTO

e) Black, white, silver only; mean and sd of price by year:

```
bws = usedcars[usedcars["color"].isin(["Black", "White", "Silver"])]
bws.groupby("year")["price"].agg(["mean", "std"])
```

	year	mean	std
2003	7900.000000	NaN	
2004	6974.500000	34.648232	
2006	8397.800000	1420.107989	
2007	9351.800000	1460.319040	
2008	10687.750000	1691.255641	
2009	13416.894737	1457.404195	
2010	14687.000000	1474.433578	
2011	16316.222222	1585.033894	
2012	17500.000000	NaN	

Years with only one car have `std = NaN` (a standard deviation of one value is undefined).

By year AND model: pass a **list** to `groupby` and save the result:

```
by_ym = bws.groupby(["year", "model"])["price"].agg(["mean", "std"])
print(by_ym.shape)
by_ym.head(4)
```

```
(19, 2)
```

	year	model	mean	std
2003	SES	7900.000000	NaN	
2004	SE	6999.000000	NaN	
	SES	6950.000000	NaN	
2006	SE	9264.666667	640.26583	

Two grouping columns give a **two-level row index** (year, model).

f) New columns by color (transform):

```
usedcars['mean_price_by_color'] = usedcars.groupby('color')['price'].transform('mean')
usedcars['count_by_color'] = usedcars.groupby('color')['price'].transform('count')
usedcars[['color', 'price', 'mean_price_by_color', 'count_by_color']].head(3)
```

	color	price	mean_price_by_color	count_by_color
0	Yellow	21992	11630.00000	3
1	Gray	20995	13665.50000	16
2	Silver	19995	13615.96875	32

12. From R to pandas

Task	R (tidyverse)	pandas
Columns by name	<code>select(attend, opponent)</code>	<code>df[["attend", "opponent"]]</code>
Filter rows	<code>filter(attend >= 40000)</code>	<code>df[df["attend"] >= 40000]</code>
Value in a set	<code>x %in% c("a", "b")</code>	<code>df["x"].isin(["a", "b"])</code>
Not in a set	<code>!(x %in% ...)</code>	<code>~df["x"].isin(...)</code>
AND / OR	<code>&, </code>	<code>&, with parentheses</code>
Sort	<code>arrange(attend), arrange(desc(attend))</code>	<code>.sort_values("attend"), ascending=False</code>
Summary by group	<code>group_by(g) %>% summarise(m = mean(x))</code>	<code>.groupby("g").agg(m=("x", "mean"))</code>
Group stat on every row	<code>group_by(g) %>% mutate(m = mean(x))</code>	<code>.groupby("g")["x"].transform("mean")</code>
New column	<code>mutate(new = ...)</code>	<code>df["new"] = ...</code>
Count of missing	<code>sum(is.na(x))</code>	<code>df["x"].isna().sum()</code>
Distinct values	<code>n_distinct(x)</code>	<code>df["x"].nunique()</code>
Counts by category	<code>count(x)</code>	<code>df["x"].value_counts()</code>
Summary of all columns	<code>summary(df)</code>	<code>df.describe()</code>
Row numbers reset	(not needed)	<code>.reset_index(drop=True)</code>

Key differences to keep straight: pandas methods usually **return a new object** (nothing changes until you assign or use `inplace=True`), **column names are quoted strings**, filtering **keeps the old row labels**, and Python numbering starts at **0** with **exclusive slice stops**.

13. Trap checklist (read before the quiz)

- `.sort_values(...)` with no assignment: original unchanged. `inplace=True` returns **None**.
- `.fillna(...)`, `.astype(...)`, `.reset_index(...)`, `.str.xxx`: all return new objects; original unchanged unless assigned.
- Filtering keeps **original index labels**; `reset_index(drop=True)` renumbers.
- `["a", "b"]` inside brackets needs to be a **list**: `df[["a", "b"]]`, not `df["a", "b"]`.
- `&`, `|`, `~` on Series with **parentheses**; never `and` / `or` / `not`.
- `iloc[a:b]` excludes `b`; columns by position; `iloc` ignores labels.
- `groupby(...).mean("attend")` averages **all numeric columns**. Pick the column with `["attend"]` *before* the function.
- `agg` collapses (one row per group); `transform` keeps the original length.
- `count()` = non-missing, `size()` = all rows, `nunique()` = distinct non-missing, `isna().sum()` = missing.
- `NaN` is skipped by `mean`, `sum`, `count`, `nunique`, `describe`; `NaN` is sorted **last**; `NaN > 5` is `False`.
- Single brackets `["col"]` give a Series; double `[["col"]]` give a DataFrame (changes the look of `agg` output).
- `str.contains` is case-sensitive, `regex (| = OR)`, and needs `.str`.
- `astype(int)` fails on `NaN`; columns with missing numbers are `float64`.
- `savefig` overwrites without warning.
- Named aggregation: `new_name=("column", "function")` - `keep =` after the new name, and the pair in parentheses.

14. Quick reference

```
df = pd.read_csv(path_or_url)
df.head(n); df.info(); df.dtypes; df.shape; df.columns; df.describe()
df["c"]; df[["c1", "c2"]]; df.iloc[rows, cols]
df["c"].min(); .max(); .median(); .mean(); .agg(["mean", "median"])
df["c"].value_counts(); df.nunique(); df.isna().sum()
df.sort_values("c", ascending=False)
df[df["c"] >= 10]; df[df["c"].isin(...)]; df[~df["c"].isin(...)]
df[(cond1) & (cond2)]; df[(cond1) | (cond2)]
df.reset_index(drop=True)
df.groupby("g")["c"].mean()
df.groupby("g")["c"].agg(["mean", "median"])
df.groupby("g").agg(new1=("c1", "mean"), new2=("c2", "max"))
df.groupby("g")["c"].transform("mean")
df["c"].fillna(df["c"].mean()); df.dropna()
df["new"] = df["s"].str.contains("a|b").astype(int)
sns.histplot(data=df, x="c", binwidth=5000); sns.boxplot(data=df, x="g", y="c"); plt.show()
```